



CYBERSTER TASK REPORT

NAME: Grace Ajayi

TRACK: Blue team

PHASE: Phase one – SOC operations and SIEM foundation

WEEK: Week 1 (Day 1 to 7)

DAY 1 – Networking fundamentals for SOC

Introduction:

On the first day, I reviewed the basic networking concepts that are important to a Security Operation Center (SOC) analyst. The goal was to understand how data travel across a network, this helps analyst interpret security alerts and detect suspicious activity.

TCP/IP and OSI Model

The **OSI model** consists of seven layers that describe how data travels across a network:

1. Physical Layer
2. Data Link Layer
3. Network Layer
4. Transport Layer
5. Session Layer
6. Presentation Layer
7. Application Layer

The **TCP/IP model**, which is widely used in real-world networking, simplifies this into four layers:

- Network Interface
- Internet
- Transport
- Application

When a user sends a web request from a browser, the request passes through these layers before reaching the server, and the response travels back through the same process.

Subnetting and CIDR

Subnetting allows networks to be divided into smaller segments to improve performance and security.

CIDR (Classless Inter-Domain Routing) notation represents subnet masks using a suffix such as /24.

Example: 192.168.56.0/24

This indicates that the first 24 bits represent the network portion of the address. Devices within the same subnet can communicate directly without requiring a router.

Common Ports and Protocols

Several common ports and protocols frequently appear in SOC investigations:

Port	Protocol	Description
22	SSH	Secure remote access
80	HTTP	Web traffic
443	HTTPS	Secure web traffic

tcp.port==443									
No.	Time	Source	Destination	Protocol	Length	Info			
2286	69.338458900	142.250.1.10	192.168.8.102	TLsv1.2	93	Application Data			
2291	69.380823900	192.168.8.102	142.250.1.10	TCP	54	63149 → 443 [ACK] Seq=272 Ack=40 Win=250 Len=0			
2292	69.434119600	142.250.1.10	192.168.8.102	TLsv1.2	123	Application Data			
2293	69.434119600	142.250.1.10	192.168.8.102	TLsv1.2	85	Application Data			
2294	69.434275800	192.168.8.102	142.250.1.10	TCP	54	63149 → 443 [ACK] Seq=272 Ack=140 Win=255 Len=0			
2295	69.434398700	142.250.1.10	192.168.8.102	TLsv1.2	93	Application Data			
2296	69.436104700	192.168.8.102	142.250.1.10	TLsv1.2	93	Application Data			
2304	69.551224800	192.168.8.102	57.140.79.12	TLsv1.2	122	Application Data			
2305	69.574941400	142.250.1.10	192.168.8.102	TCP	54	443 → 63149 [ACK] Seq=179 Ack=311 Win=1045 Len=0			
2306	69.584272100	57.140.79.12	192.168.8.102	TCP	54	443 → 60491 [ACK] Seq=211 Ack=273 Win=591 Len=0			
2307	69.644971700	192.168.8.102	108.162.106.42	TLsv1.2	104	Application Data			
2309	69.715435200	57.140.79.12	192.168.8.102	TLsv1.2	124	Application Data			
2311	69.769262300	192.168.8.102	57.140.79.12	TCP	54	60491 → 443 [ACK] Seq=273 Ack=281 Win=255 Len=0			
2312	69.780359300	108.162.106.42	192.168.8.102	TLsv1.2	93	Application Data			
2315	69.831165600	192.168.8.102	108.162.106.42	TCP	54	62399 → 443 [ACK] Seq=101 Ack=79 Win=253 Len=0			
2320	70.859095600	20.215.79.200	192.168.8.102	TLsv1.2	81	Application Data			
2323	70.905495000	192.168.8.102	20.215.79.200	TCP	54	60153 → 443 [ACK] Seq=181 Ack=136 Win=253 Len=0			
2355	71.853668300	192.168.8.102	52.146.170.13	TLsv1.2	82	Application Data			
2375	72.004982500	52.146.170.13	192.168.8.102	TCP	54	443 → 56767 [ACK] Seq=136 Ack=209 Win=608 Len=0			
2376	72.009160300	192.168.8.102	20.215.79.200	TLsv1.2	82	Application Data			
2378	72.145484600	20.215.79.200	192.168.8.102	TCP	54	443 → 60153 [ACK] Seq=136 Ack=209 Win=608 Len=0			
2379	72.164800900	192.168.8.102	52.146.170.13	TLsv1.2	85	Application Data			
2380	72.300950300	52.146.170.13	192.168.8.102	TCP	54	443 → 56767 [ACK] Seq=136 Ack=240 Win=608 Len=0			
2381	72.353878400	57.140.79.230	192.168.8.102	TLsv1.2	456	Application Data			
2382	72.354429300	192.168.8.102	57.140.79.230	TLsv1.2	1349	Application Data			
2383	72.544057200	57.140.79.230	192.168.8.102	TCP	54	443 → 63677 [ACK] Seq=805 Ack=2591 Win=16387 Len=0			
2388	73.065987600	192.168.8.102	52.146.170.1216	TLsv1.2	112	Application Data			
2390	73.314633700	52.146.170.1216	192.168.8.102	TLsv1.2	101	Application Data			

```
Frame 2315: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{B43838E3-C9CC-4EC4-93D4-82CBA}
Ethernet II, Src: Intel d:c:d:a:b6 (28:16:ad:d:c:d:a:b6), Dst: NetXtreme c7:b2:6a (e4:7d:eb:c7:b2:6a)
Internet Protocol Version 4, Src: 192.168.8.102, Dst: 108.162.106.42
Transmission Control Protocol, Src Port: 62399, Dst Port: 443, Seq: 101, Ack: 79, Len: 0
```

- TCP handshakes

tcp									
No.	Time	Source	Destination	Protocol	Length	Info			
2890	90.144509600	52.84.151.20	10.145.67.112	TCP	54	601 → 61307 [ACK] Seq=1838 Ack=486 Win=600 Len=0			
2891	90.145152900	52.84.151.20	10.145.67.112	TLsv1.2	93	Application Data			
2892	90.147259600	52.84.151.20	10.145.67.112	TCP	140	601 → 61307 [ACK] Seq=1837 Ack=486 Win=163 Len=1389 [TCP PMTU recommended by 3894]			
2893	90.147328100	10.145.67.112	52.84.151.20	TCP	54	61307 → 601 [ACK] Seq=686 Ack=1837 Win=600 Len=0			
2894	90.148569400	52.84.151.20	10.145.67.112	TLsv1.2	472	Application Data			
2912	90.194085100	10.145.67.112	52.84.151.20	TCP	54	61367 → 443 [WIN] Seq=406 Ack=3675 Win=100 Len=0			
2917	90.273476800	10.145.67.112	20.215.79.68	TCP	54	62961 → 443 [FIN, ACK] Seq=140 Ack=60000 Len=0			
2924	90.307239800	10.145.67.112	10.145.67.112	TLsv1.3	1018	Application Data, Application Data			
2932	90.351670400	10.145.67.112	64.233.185.139	TCP	54	14010 → 800 [FIN] Seq=2890 Ack=7647 Win=41000 Len=0			
2940	90.400737700	10.145.67.112	10.145.67.112	TCP	54	443 → 62961 [FIN, ACK] Seq=140 Ack=60000 Len=0			
2949	90.408818100	10.145.67.112	20.190.109.68	TCP	54	60960 → 800 [ACK] Seq=0 Ack=0 Win=0 Len=0			
2951	90.554352000	10.145.67.112	74.125.138.84	TCP	55	[TCP Keep-Alive] 60960 → 800 [ACK] Seq=0 Ack=0 Win=0 Len=0			
2958	90.751021300	10.145.67.112	10.145.67.112	TCP	66	[TCP Keep-Alive] 60960 → 800 [ACK] Seq=0 Ack=0 Win=0 Len=0			
2987	90.901328500	10.145.67.112	14.189.147.91	TCP	66	63818 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1280 SACK_PERM WS=2			
2988	90.907415600	10.145.67.112	14.189.147.91	TCP	66	63819 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1280 SACK_PERM WS=2			
2989	91.020260800	10.145.67.112	14.189.147.91	TCP	66	63820 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1280 SACK_PERM WS=2			
2992	91.020357400	10.145.67.112	140.154.167.91	TCP	54	63818 → 800 [ACK] Seq=1 Ack=1 Win=65200 Len=0			
2996	91.027260800	10.145.67.112	140.154.167.91	SSL	183	Continuation Data			
2997	91.034276400	140.154.167.91	10.145.67.112	TCP	70	800 → 63818 [WIN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1280 SACK_PERM WS=2			
2998	91.034375600	10.145.67.112	140.154.167.91	TCP	54	63818 → 800 [ACK] Seq=1 Ack=1 Win=65200 Len=0			
2999	91.035112800	10.145.67.112	140.154.167.91	TCP	278	800 → 63818 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0 [TCP PMTU recommended by 3849]			
3044	91.169984300	140.154.167.91	10.145.67.112	SSL	143	Continuation Data			
3049	91.171454000	10.145.67.112	140.154.167.91	HTTP	102	POST / HTTP/1.1 (application/x-www-form-urlencoded)			
3050	91.174279000	10.145.67.112	140.154.167.91	SSL	301	Continuation Data			
3062	91.207215800	140.154.167.91	10.145.67.112	TCP	60	800 → 63818 [ACK] Seq=1 Ack=1 Win=65535 Len=0			
3068	91.206230000	140.154.167.91	10.145.67.112	TCP	60	800 → 63818 [ACK] Seq=1 Ack=1 Win=65535 Len=0			
3069	91.206335800	14.189.147.91	140.154.167.91	TCP	54	63820 → 800 [ACK] Seq=1 Ack=1 Win=65535 Len=0			
3070	91.207300000	14.189.147.91	140.154.167.91	TCP	54	63820 → 800 [ACK] Seq=1 Ack=1 Win=65535 Len=0			

Frame 2988: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{B43838E3-C9CC-4EC4-93D4-82CBA}

Ethernet II, Src: 42:c1:67:c7:a0:00 (08:00:27:00:00:00), Dst: 06:f4:34:06:18:71 (00:14:34:06:18:71)

Internet Protocol Version 4, Src: 10.145.67.112, Dst: 14.189.147.91

Transmission Control Protocol, Src Port: 63818, Dst Port: 80, Seq: 63818, Len: 0

0000 06 f4 34 06 18 71 c7 c1 67 c7 a0 0

- ICMP echo requests

No.	Time	Source	Destination	Protocol	Length	Info			
2317	69.849715400	192.168.8.102	192.168.8.102	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 2318)			
2318	69.877970500	192.168.8.102	192.168.8.102	ICMP	74	Echo (ping) reply id=0x0001, seq=3/768, ttl=128 (request in 2317)			
2321	70.859574700	192.168.8.102	192.168.8.102	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in 2322)			
2322	70.899422100	192.168.8.102	192.168.8.102	ICMP	74	Echo (ping) reply id=0x0001, seq=4/1024, ttl=128 (request in 2321)			
2356	71.868917700	192.168.8.102	192.168.8.102	ICMP	74	Echo (ping) request id=0x0001, seq=5/1280, ttl=128 (reply in 2373)			
2373	71.936175100	192.168.8.102	192.168.8.102	ICMP	74	Echo (ping) reply id=0x0001, seq=5/1280, ttl=128 (request in 2356)			
2386	72.079005700	192.168.8.102	192.168.8.102	ICMP	74	Echo (ping) request id=0x0001, seq=6/1536, ttl=128 (reply in 2387)			
2387	72.904956800	192.168.8.102	192.168.8.102	ICMP	74	Echo (ping) reply id=0x0001, seq=6/1536, ttl=128 (request in 2386)			

- ARP requests within the network

No.	Time	Source	Destination	Protocol	Length	Info
298	22.097928000	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
299	22.097961800	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
686	47.138382800	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
687	47.138415000	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
1498	71.455433400	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
1499	71.455465800	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
4247	95.010683000	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
4248	95.010722300	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
6246	118.151129800	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
6247	118.151168300	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
6558	144.157736100	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
6559	144.157772500	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
7285	167.709317100	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
7286	167.709358400	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
7315	174.204065900	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	Who has 10.145.67.194? Tell 10.145.67.112
7316	174.208911600	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	10.145.67.194 is at 06:f4:34:06:18:71
7502	200.167997700	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
7503	200.168045200	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
8094	248.809269300	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
8095	248.809373500	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
9334	273.383301600	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
9336	273.383326800	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
11450	290.026785000	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
11451	290.026818800	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
11571	308.795180900	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194
11572	308.795228000	42:c1:67:c7:a0:9f	06:f4:34:06:18:71	ARP	42	10.145.67.112 is at 42:c1:67:c7:a0:9f
11839	325.094156500	06:f4:34:06:18:71	42:c1:67:c7:a0:9f	ARP	42	Who has 10.145.67.112? Tell 10.145.67.194

Frame 298: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{B43830E3-49CC-4EC4-93D4-82CBA7} Ethernet II, Src: 06:f4:34:06:18:71 (06:f4:34:06:18:71), Dst: 42:c1:67:c7:a0:9f (42:c1:67:c7:a0:9f) Address Resolution Protocol (request)

000042c167c7a09f06f43406001008000004000100f434060020000000000000a914370

This exercise demonstrated how network traffic appears during normal activity.

Day 2 — VMware Lab Setup

On Day 2, a virtual security lab environment was created using **VMware Workstation** instead of VirtualBox.

Virtual Machines Created

Three virtual machines were deployed:

VM	Purpose	OS
Kali Linux	Attack simulation	Kali Linux
Ubuntu Server	SIEM server	Ubuntu Server
Windows 10	Monitored endpoint	Windows 10

Resource Allocation

VM	CPU	RAM	Storage
Kali Linux	4 cores	2GB	80 GB
Ubuntu Server	2 cores	4 GB	50 GB
Windows 10	2 cores	2 GB	60 GB

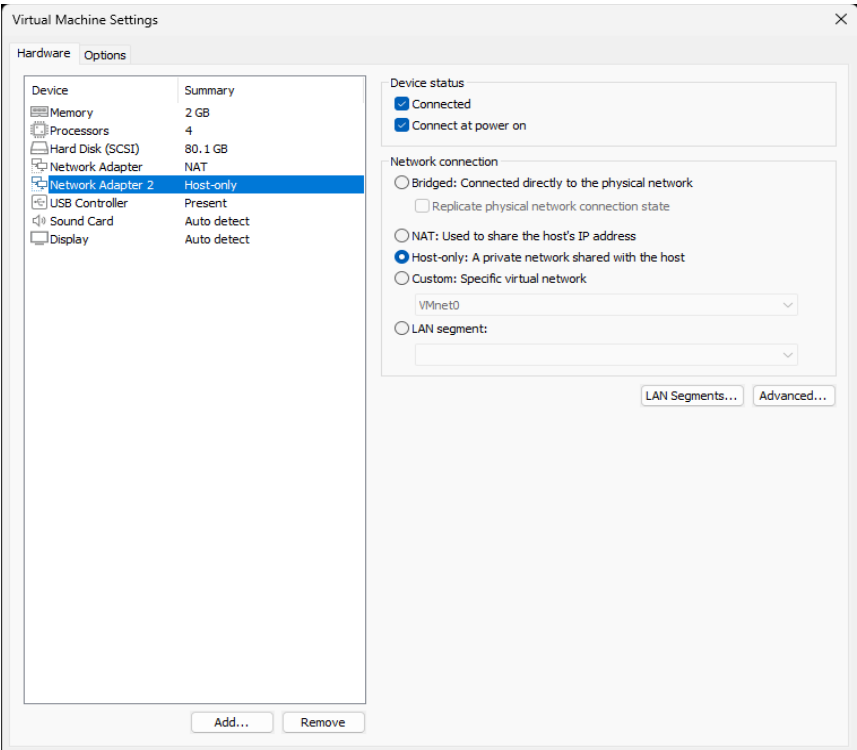


Fig1: VMware network configuration for Kali Linux showing both NAT and Host-Only adapters enabled.

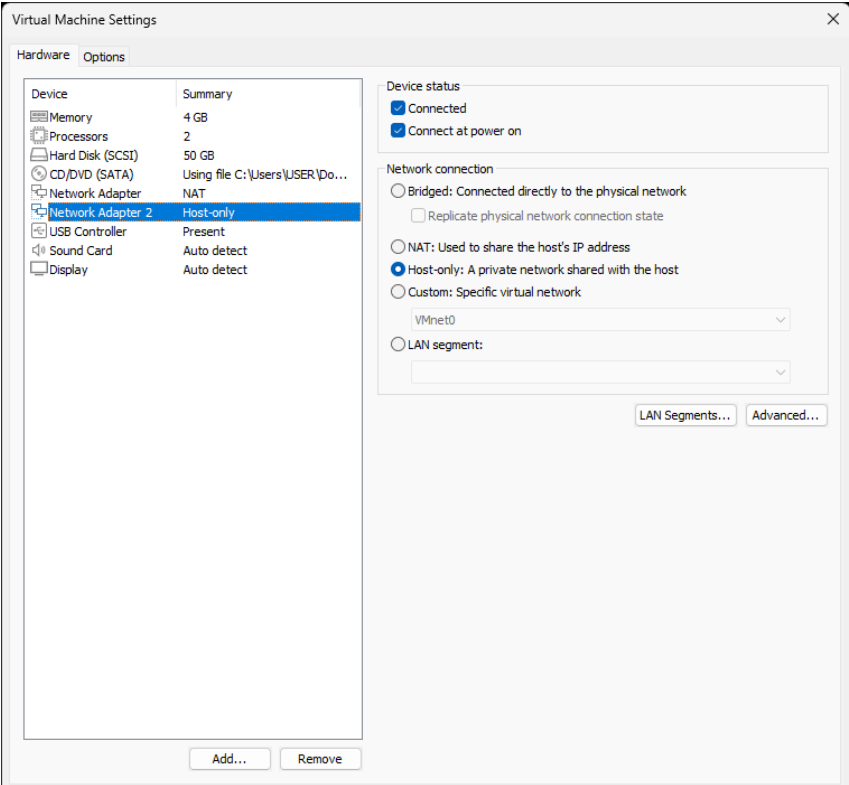


Fig2: VMware network configuration for Ubuntu showing both NAT and Host-Only adapters enabled.

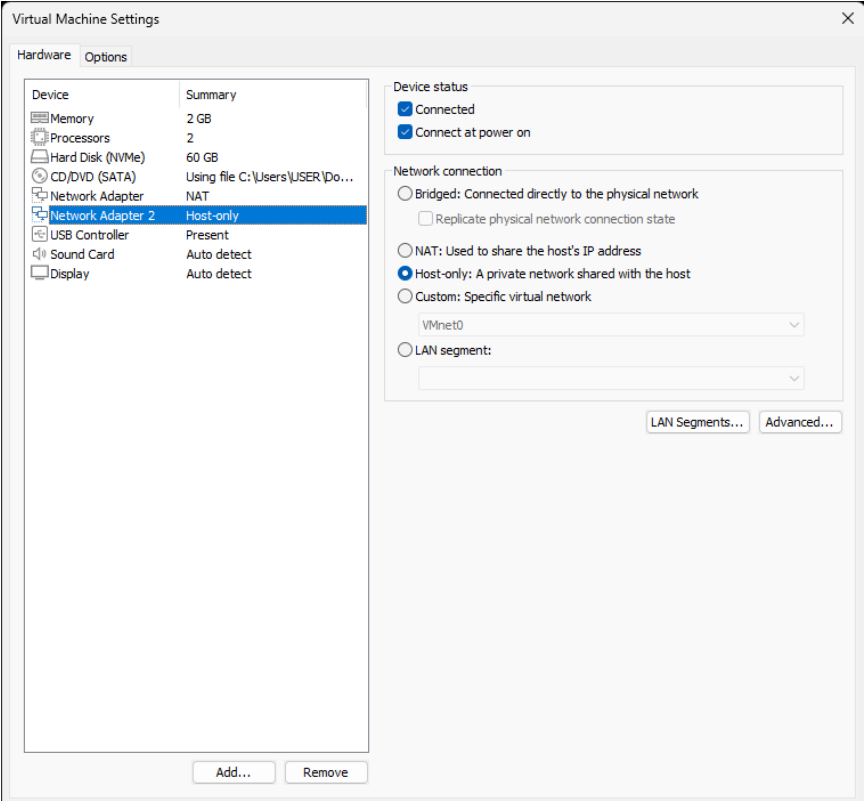


Fig3: VMware network configuration for Windows10 showing both NAT and Host-Only adapters enabled.

Network Configuration

Each VM was configured with two network adapters:

- 1. Host-Only Adapter
- 2. NAT Adapter

1. **Host-Only Network:**

This network allows communication between the virtual machines and the host system but isolates them from the external network.

2. **NAT Network:**

This adapter provides internet connectivity to allow the VMs to download updates and install required software. This configuration ensures the lab remains isolated while still allowing necessary internet access.

Connectivity Testing:

After booting the virtual machines, IP addresses were verified using: -ip a and -ipconfig.

```
Session Actions Edit View Help
(Gracey@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKN
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
    valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_code
link/ether 00:0c:29:a1:ab:96 brd ff:ff:ff:ff:ff:ff
inet 192.168.11.129/24 brd 192.168.11.255 scope global dynami
    valid_lft 1406sec preferred_lft 1406sec
inet6 fe80::8104:4ea9:48ee:de78/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_code
link/ether 00:0c:29:a1:ab:a0 brd ff:ff:ff:ff:ff:ff
inet 192.168.6.128/24 brd 192.168.6.255 scope global dynamic
    valid_lft 1225sec preferred_lft 1225sec
inet6 fe80::a078:bac7:979f:8a20/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
```

Fig1: Network interface configuration on Kali Linux showing both NAT and Host-Only adapters with assigned IP addresses.

```
gracey@ubuntu20.04:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
    valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 00:0c:29:e5:98:e6 brd ff:ff:ff:ff:ff:ff
altname enp2s1
inet 192.168.11.131/24 metric 100 brd 192.168.11.255 scope global dynamic ens33
    valid_lft 1624sec preferred_lft 1624sec
inet6 fe80::20c:29ff:fee5:98e6/64 scope link
    valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 00:0c:29:e5:98:f0 brd ff:ff:ff:ff:ff:ff
altname enp2s5
inet 192.168.6.130/24 brd 192.168.6.255 scope global dynamic ens37
    valid_lft 1792sec preferred_lft 1792sec
inet6 fe80::20c:29ff:fee5:98f0/64 scope link
    valid_lft forever preferred_lft forever
```

Fig2: Network interface configuration on Ubuntu showing both NAT and Host Only adapters with assigned IP addresses.


```

Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Analyst>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::c98c:2914:1400:4000
    IPv4 Address. . . . . : 192.168.6.128
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.6.1

Ethernet adapter Ethernet1:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::f86:d1:1400:4000
    IPv4 Address. . . . . : 192.168.6.129
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

C:\Users\Analyst>
```

Fig3: Network interface configuration on Kali Linux showing both NAT and Host-Only adapters with assigned IP addresses.

Ping tests were conducted between all machines using their Host-Only IP addresses.

Successful ICMP ping tests between virtual machines confirming network connectivity across the Host-Only network;

```

(Gracey@kali)-[~]
$ ping 192.168.6.129
PING 192.168.6.129 (192.168.6.129) 56(84) bytes of data.
 64 bytes from 192.168.6.129: icmp_seq=1 ttl=128 time=3.55 ms
 64 bytes from 192.168.6.129: icmp_seq=2 ttl=128 time=2.34 ms
 64 bytes from 192.168.6.129: icmp_seq=3 ttl=128 time=2.30 ms
 64 bytes from 192.168.6.129: icmp_seq=4 ttl=128 time=3.09 ms
 64 bytes from 192.168.6.129: icmp_seq=5 ttl=128 time=3.22 ms
 64 bytes from 192.168.6.129: icmp_seq=6 ttl=128 time=2.70 ms
^C
--- 192.168.6.129 ping statistics ---
 6 packets transmitted, 6 received, 0% packet loss, time 5021ms
 rtt min/avg/max/mdev = 2.296/2.864/3.547/0.460 ms

(Gracey@kali)-[~]
$ ping 192.168.6.130
PING 192.168.6.130 (192.168.6.130) 56(84) bytes of data.
 64 bytes from 192.168.6.130: icmp_seq=1 ttl=64 time=6.52 ms
 64 bytes from 192.168.6.130: icmp_seq=2 ttl=64 time=0.879 ms
 64 bytes from 192.168.6.130: icmp_seq=3 ttl=64 time=0.732 ms
 64 bytes from 192.168.6.130: icmp_seq=4 ttl=64 time=0.724 ms
 64 bytes from 192.168.6.130: icmp_seq=5 ttl=64 time=1.08 ms
 64 bytes from 192.168.6.130: icmp_seq=6 ttl=64 time=1.45 ms
^C
--- 192.168.6.130 ping statistics ---
 6 packets transmitted, 6 received, 0% packet loss, time 5018ms
 rtt min/avg/max/mdev = 0.724/1.897/6.522/2.082 ms
```

Fig4: Kali sending a ping request to ubuntu and windows

```

gracey@ubuntu-wazuh:~$ ping 192.168.6.128
PING 192.168.6.128 (192.168.6.128) 56(84) bytes of data.
 64 bytes from 192.168.6.128: icmp_seq=1 ttl=64 time=0.768 ms
 64 bytes from 192.168.6.128: icmp_seq=2 ttl=64 time=0.704 ms
 64 bytes from 192.168.6.128: icmp_seq=3 ttl=64 time=0.643 ms
 64 bytes from 192.168.6.128: icmp_seq=4 ttl=64 time=0.627 ms
 64 bytes from 192.168.6.128: icmp_seq=5 ttl=64 time=0.765 ms
^C
--- 192.168.6.128 ping statistics ---
 5 packets transmitted, 5 received, 0% packet loss, time 4032ms
 rtt min/avg/max/mdev = 0.627/0.701/0.768/0.059 ms
gracey@ubuntu-wazuh:~$ ping 192.168.6.129
PING 192.168.6.129 (192.168.6.129) 56(84) bytes of data.
 64 bytes from 192.168.6.129: icmp_seq=1 ttl=128 time=0.992 ms
 64 bytes from 192.168.6.129: icmp_seq=2 ttl=128 time=0.892 ms
 64 bytes from 192.168.6.129: icmp_seq=3 ttl=128 time=4.03 ms
 64 bytes from 192.168.6.129: icmp_seq=4 ttl=128 time=2.59 ms
 64 bytes from 192.168.6.129: icmp_seq=5 ttl=128 time=1.22 ms
^C
--- 192.168.6.129 ping statistics ---
 5 packets transmitted, 5 received, 0% packet loss, time 4004ms
 rtt min/avg/max/mdev = 0.892/1.944/4.030/1.209 ms
gracey@ubuntu-wazuh:~$ _
```

Fig5: Ubuntu sending a ping request to kali and windows

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Analyst>ping 192.168.6.128

Pinging 192.168.6.128 with 32 bytes of data:
Reply from 192.168.6.128: bytes=32 time<1ms TTL=64
Reply from 192.168.6.128: bytes=32 time<1ms TTL=64
Reply from 192.168.6.128: bytes=32 time<1ms TTL=64
Reply from 192.168.6.128: bytes=32 time=3ms TTL=64

Ping statistics for 192.168.6.128:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 3ms, Average = 0ms

C:\Users\Analyst>ping 192.168.6.130

Pinging 192.168.6.130 with 32 bytes of data:
Reply from 192.168.6.130: bytes=32 time<1ms TTL=64
Reply from 192.168.6.130: bytes=32 time<1ms TTL=64
Reply from 192.168.6.130: bytes=32 time<1ms TTL=64
Reply from 192.168.6.130: bytes=32 time=8ms TTL=64

Ping statistics for 192.168.6.130:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 8ms, Average = 2ms
```

Fig6: Windows sending a ping request to Kali and Ubuntu.

Days 3–4 — Wazuh Manager Installation

The Wazuh SIEM platform was installed on the Ubuntu Server VM.

Wazuh consists of three primary components:

- 1. **Wazuh Manager** – Processes security events and applies detection rules
- 2. **Wazuh Indexer** – Stores log data and alerts
- 3. **Wazuh Dashboard** – Web interface for security monitoring

Static IP Configuration

A static IP address was already assigned to the Ubuntu Server Host-Only interface to ensure agents could consistently connect to the Wazuh Manager.

Example: 192.168.6.130

Installation Process

The Wazuh QuickStart installation script from the official documentation was used.

Example command: curl -sO <https://packages.wazuh.com/4.7/wazuh-install.sh>

Sudo bash wazuh-install.sh -a -i

The installation automatically configured the manager, indexer, and dashboard components.

```
gracey@ubuntu:wazuh:~$ sudo ./wazuh-install.sh -a -i
[sudo] password for gracey:
24/03/2026 22:18:51 INFO: Starting Wazuh installation assistant. Wazuh version: 4.7.5
24/03/2026 22:18:51 INFO: Verbose logging redirected to /var/log/wazuh-install.log
24/03/2026 22:19:12 WARNING: Hardware and system checks ignored.
24/03/2026 22:19:12 INFO: Wazuh web interface port will be 443.
24/03/2026 22:19:27 INFO: Wazuh repository added.
24/03/2026 22:19:27 INFO: --- Configuration files ---
24/03/2026 22:19:27 INFO: Generating configuration files.
24/03/2026 22:19:32 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
24/03/2026 22:19:32 INFO: --- Wazuh indexer ---
24/03/2026 22:19:32 INFO: Starting Wazuh indexer installation.
24/03/2026 22:22:47 INFO: Wazuh indexer installation finished.
24/03/2026 22:22:48 INFO: Wazuh indexer post-install configuration finished.
24/03/2026 22:22:48 INFO: Starting service wazuh-indexer.
24/03/2026 22:23:29 INFO: wazuh-indexer service started.
24/03/2026 22:23:29 INFO: Initializing Wazuh indexer cluster security settings.
24/03/2026 22:23:40 INFO: Wazuh indexer cluster initialized.
24/03/2026 22:23:40 INFO: --- Wazuh server ---
24/03/2026 22:23:40 INFO: Starting the Wazuh manager installation.
24/03/2026 22:29:08 INFO: Wazuh manager installation finished.
24/03/2026 22:29:08 INFO: Starting service wazuh-manager.
24/03/2026 22:29:37 INFO: wazuh-manager service started.
24/03/2026 22:29:37 INFO: Starting Filebeat installation.
24/03/2026 22:30:08 INFO: Filebeat installation finished.
24/03/2026 22:30:11 INFO: Filebeat post-install configuration finished.
24/03/2026 22:30:11 INFO: Starting service filebeat.
24/03/2026 22:30:14 INFO: filebeat service started.
24/03/2026 22:30:14 INFO: --- Wazuh dashboard ---
24/03/2026 22:30:14 INFO: Starting Wazuh dashboard installation.
24/03/2026 22:33:17 INFO: Wazuh dashboard installation finished.
24/03/2026 22:33:17 INFO: Wazuh dashboard post-install configuration finished.
24/03/2026 22:33:17 INFO: Starting service wazuh-dashboard.
24/03/2026 22:33:19 INFO: wazuh-dashboard service started.
24/03/2026 22:34:16 INFO: Initializing Wazuh dashboard web application.
24/03/2026 22:34:17 INFO: Wazuh dashboard web application initialized.
24/03/2026 22:34:17 INFO: --- Summary ---
24/03/2026 22:34:17 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
User: admin
Password: /
24/03/2026 22:34:17 INFO: Installation finished.
gracey@ubuntu:wazuh:~$
```

Service Verification

After installation, services were verified using:

sudo systemctl status Wazuh-manager

sudo systemctl status Wazuh-indexer

sudo systemctl status Wazuh-dashboard

```
ubuntu@ubuntu:~$ sudo systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-04-08 12:03:58 UTC; 1h 1min ago
     Process: 986 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 121 (limit: 4544)
   Memory: 318.7M (peak: 573.2M swap: 15.9M swap peak: 15.9M)
      CPU: 2min 47.384s
   CGroup: /system.slice/wazuh-manager.service
           └─1627 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             1632 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             1637 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             1647 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             1696 /var/ossec/bin/wazuh-authd
             1733 /var/ossec/bin/wazuh-db
             1787 /var/ossec/bin/wazuh-execd
             1845 /var/ossec/bin/wazuh-analysisd
             1867 /var/ossec/bin/wazuh-syscheckd
             1884 /var/ossec/bin/wazuh-remoted
             1928 /var/ossec/bin/wazuh-logcollector
             1941 /var/ossec/bin/wazuh-monitord
             1963 /var/ossec/bin/wazuh-modulesd

12:03:51 ubuntu@ubuntu:~$ wazuh-remoted: Process 1642 not used by Wazuh, removing...
12:03:52 ubuntu@ubuntu:~$ wazuh-remoted: Started wazuh-remoted...
12:03:53 ubuntu@ubuntu:~$ wazuh-logcollector: Process 1688 not used by Wazuh, removing...
12:03:54 ubuntu@ubuntu:~$ wazuh-logcollector: Started wazuh-logcollector...
12:03:54 ubuntu@ubuntu:~$ wazuh-monitord: Process 1707 not used by Wazuh, removing...
12:03:54 ubuntu@ubuntu:~$ wazuh-monitord: Started wazuh-monitord...
12:03:55 ubuntu@ubuntu:~$ wazuh-modulesd: Process 1750 not used by Wazuh, removing...
12:03:56 ubuntu@ubuntu:~$ wazuh-modulesd: Started wazuh-modulesd...
12:03:58 ubuntu@ubuntu:~$ wazuh-modulesd: Completed.
12:03:58 ubuntu@ubuntu:~$ systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-04-08 12:03:58 UTC; 1h 1min ago
     Process: 986 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 121 (limit: 4544)
   Memory: 318.7M (peak: 573.2M swap: 15.9M swap peak: 15.9M)
      CPU: 2min 47.384s
   CGroup: /system.slice/wazuh-manager.service
           └─1627 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             1632 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             1637 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             1647 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             1696 /var/ossec/bin/wazuh-authd
             1733 /var/ossec/bin/wazuh-db
             1787 /var/ossec/bin/wazuh-execd
             1845 /var/ossec/bin/wazuh-analysisd
             1867 /var/ossec/bin/wazuh-syscheckd
             1884 /var/ossec/bin/wazuh-remoted
             1928 /var/ossec/bin/wazuh-logcollector
             1941 /var/ossec/bin/wazuh-monitord
             1963 /var/ossec/bin/wazuh-modulesd
```

Fig1: Shows active running of the Wazuh Manager.

```
ubuntu@ubuntu:~$ sudo systemctl status wazuh-indexer
● wazuh-indexer.service - Wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-04-08 12:05:39 UTC; 1h 7min ago
     Docs: https://documentation.wazuh.com
    Main PID: 978 (java)
     Tasks: 86 (limit: 4544)
   Memory: 2.1G (peak: 2.3G swap: 128.5M swap peak: 128.6M)
      CPU: 5min 4.541s
   CGroup: /system.slice/wazuh-indexer.service
           └─978 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cache.ttl=60 -Dopensearch.networkaddress.cache.negative.ttl=10

12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.bootstrap.Bootstrap.setup(Bootstrap.java:242)
12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.bootstrap.Bootstrap.init(Bootstrap.java:404)
12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.bootstrap.OpenSearch.init(OpenSearch.java:180)
12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.bootstrap.OpenSearch.execute(OpenSearch.java:171)
12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.cli.EnvironmentAwareCommand.execute(EnvironmentAwareCommand.java:104)
12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.cli.Command.mainWithoutErrorHandling(Command.java:138)
12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.cli.Command.main(Command.java:101)
12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.bootstrap.OpenSearch.main(OpenSearch.java:137)
12:04:41 ubuntu@ubuntu:~$ wazuh-indexer: at org.opensearch.bootstrap.OpenSearch.main(OpenSearch.java:103)
12:05:39 ubuntu@ubuntu:~$ systemctl status wazuh-indexer
● wazuh-indexer.service - Wazuh-indexer
   Loaded: loaded (/usr/lib/systemd/system/wazuh-indexer.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-04-08 12:05:39 UTC; 1h 7min ago
     Docs: https://documentation.wazuh.com
    Main PID: 978 (java)
     Tasks: 86 (limit: 4544)
   Memory: 2.1G (peak: 2.3G swap: 128.5M swap peak: 128.6M)
      CPU: 5min 4.541s
   CGroup: /system.slice/wazuh-indexer.service
           └─978 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cache.ttl=60 -Dopensearch.networkaddress.cache.negative.ttl=10
```

Fig2: Shows active running of the Wazuh indexer.

```
ubuntu@ubuntu:~$ systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-04-08 12:03:20 UTC; 1h 17min ago
    Main PID: 845 (node)
     Tasks: 11 (limit: 4544)
   Memory: 207.8M (peak: 317.1M swap: 744.0K swap peak: 744.0K)
      CPU: 51.977s
   CGroup: /system.slice/wazuh-dashboard.service
           └─845 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandled-rejections=warn /usr/share/wazuh-dashboard/s

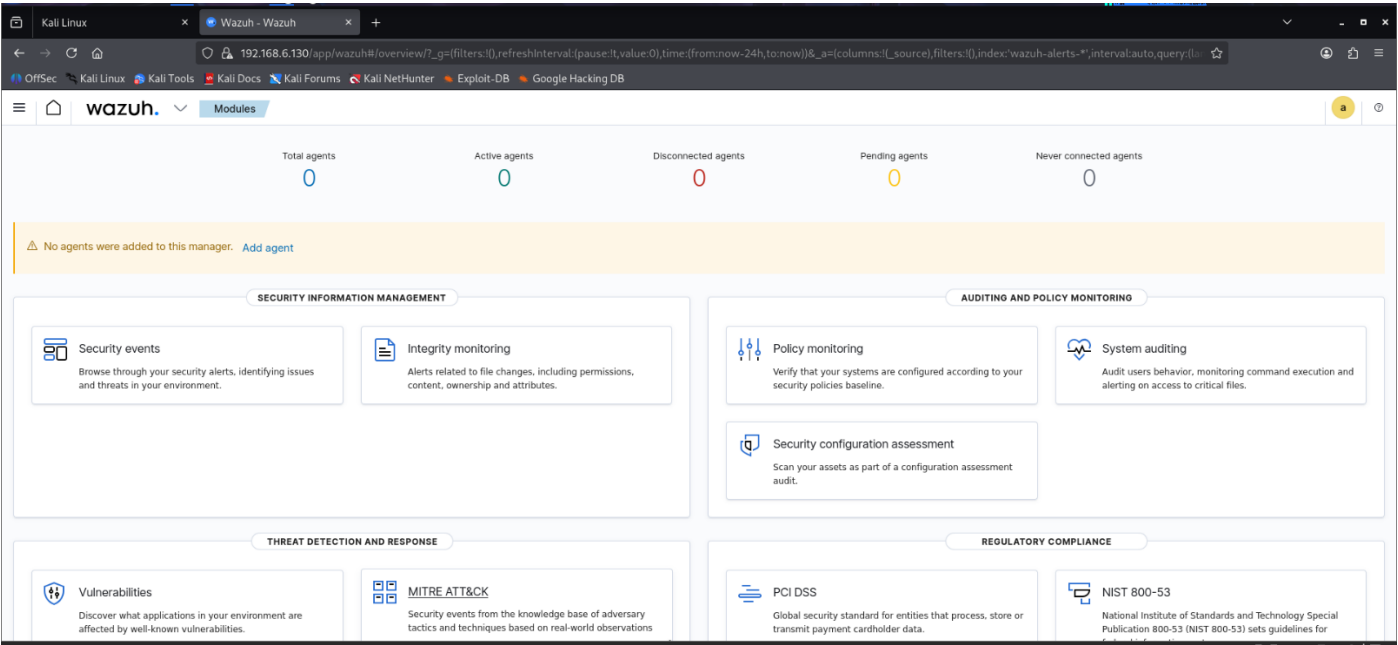
Apr 08 12:59:09 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:09Z","tags":[],"pid":845,"method":"get","statusCode":200}
Apr 08 12:59:09 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:09Z","tags":[],"pid":845,"method":"get","statusCode":200}
Apr 08 12:59:09 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:09Z","tags":[],"pid":845,"method":"post","statusCode":200}
Apr 08 12:59:10 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:10Z","tags":[],"pid":845,"method":"post","statusCode":200}
Apr 08 12:59:10 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:10Z","tags":[],"pid":845,"method":"post","statusCode":200}
Apr 08 12:59:10 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:10Z","tags":[],"pid":845,"method":"post","statusCode":200}
Apr 08 12:59:10 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:10Z","tags":[],"pid":845,"method":"post","statusCode":200}
Apr 08 12:59:10 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:10Z","tags":[],"pid":845,"method":"post","statusCode":200}
Apr 08 12:59:10 ubuntu@ubuntu:~$ wazuh-dashboard: {"type":"response","timestamp":"2026-04-08T12:59:10Z","tags":[],"pid":845,"method":"post","statusCode":200}
```

Fig 3: Shows active running of the Wazuh dashboard.

All services were confirmed running.

Dashboard Access

The Wazuh dashboard was accessed via a web browser using the Ubuntu Server IP:
https://192.168.6.130



Img: After logging in, the dashboard interface displayed security information Management, and Auditing & Policy monitoring

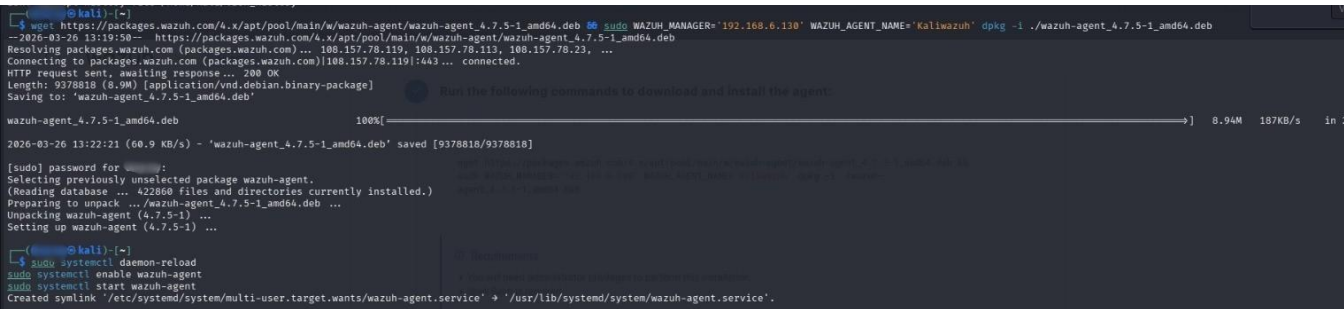
Days 5–6 — Wazuh Agent Deployment

Agents were installed on both the Kali Linux and Windows 10 virtual machines.
Agents collect log data and send it to the Wazuh Manager.

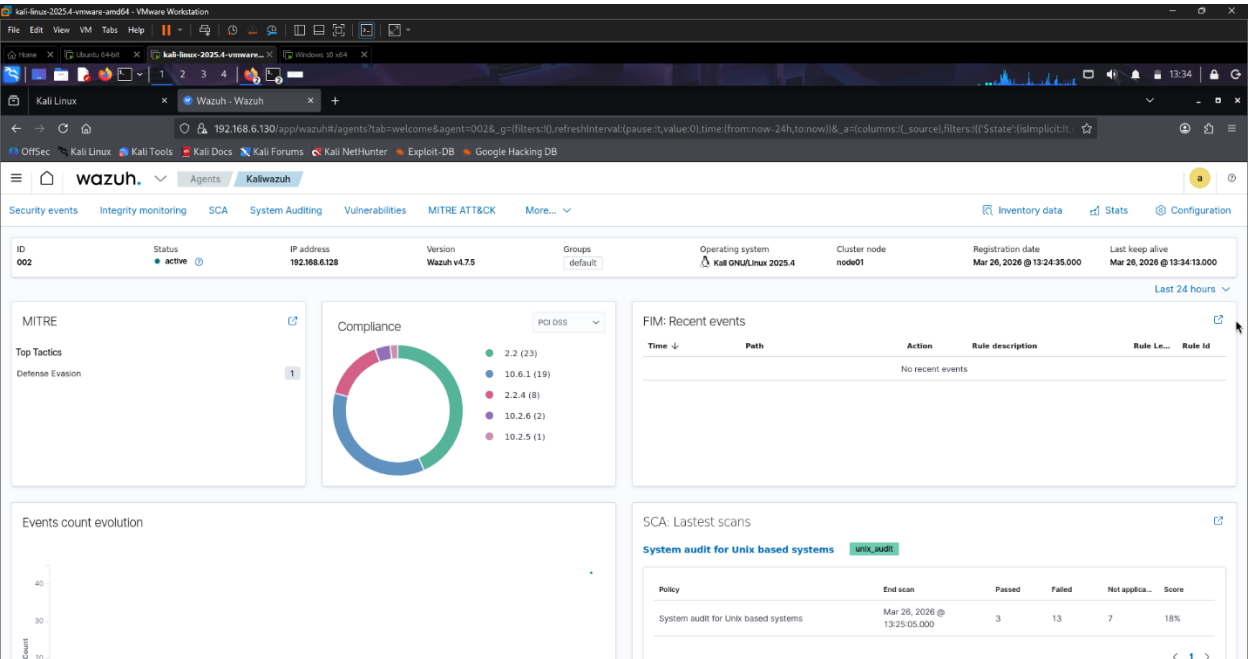
- **Kali Linux Agent Installation**

The Wazuh dashboard deployment wizard generated the correct installation command for Kali Linux.

After installation, the agent service was started and successfully connected to the Wazuh Manager.



img1: Deployment of the Wazuh agent on Kali.

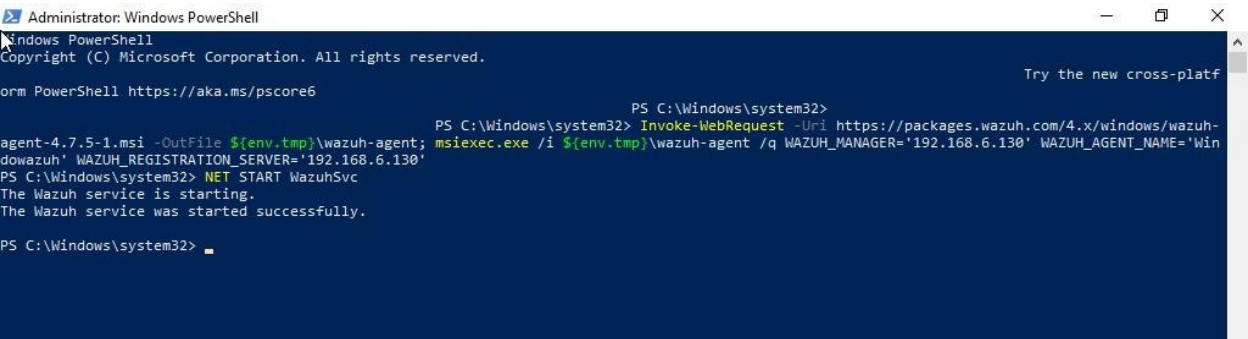


img2: Kali-agent successfully connected and running as the second agent on Wazuh.

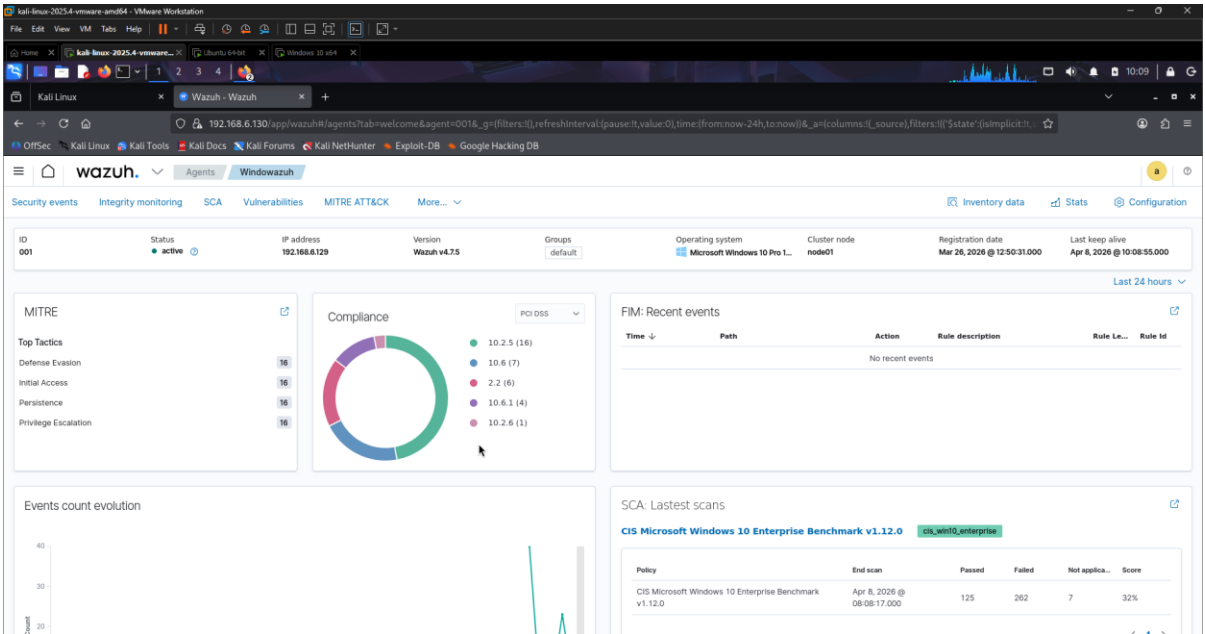
- **Windows 10 Agent Installation**

The Windows agent was installed using a PowerShell command provided by the Wazuh dashboard deployment wizard.

The agent service was started and confirmed active in the dashboard.



Img1: Deployment of Wazuh agent on windows.



img2: windows -agent successfully connected and running as the second agent on Wazuh.

Agent Configuration File

The primary configuration file used by Wazuh agents is: ossec.conf

Important sections includes;

- **Server Block:** Defines the IP address of the Wazuh Manager where logs are sent.
- **Syscheck Block:** Used for file integrity monitoring to detect unauthorized file changes.
- **Localfile Block:** Specifies system logs that the agent should monitor.

Test Alert Generation

To confirm the pipeline was functioning correctly, several failed SSH login attempts were generated from Kali Linux to Ubuntu Server.

Example command: ssh user@192.168.56.10

Multiple incorrect password attempts triggered alerts in the Wazuh dashboard.

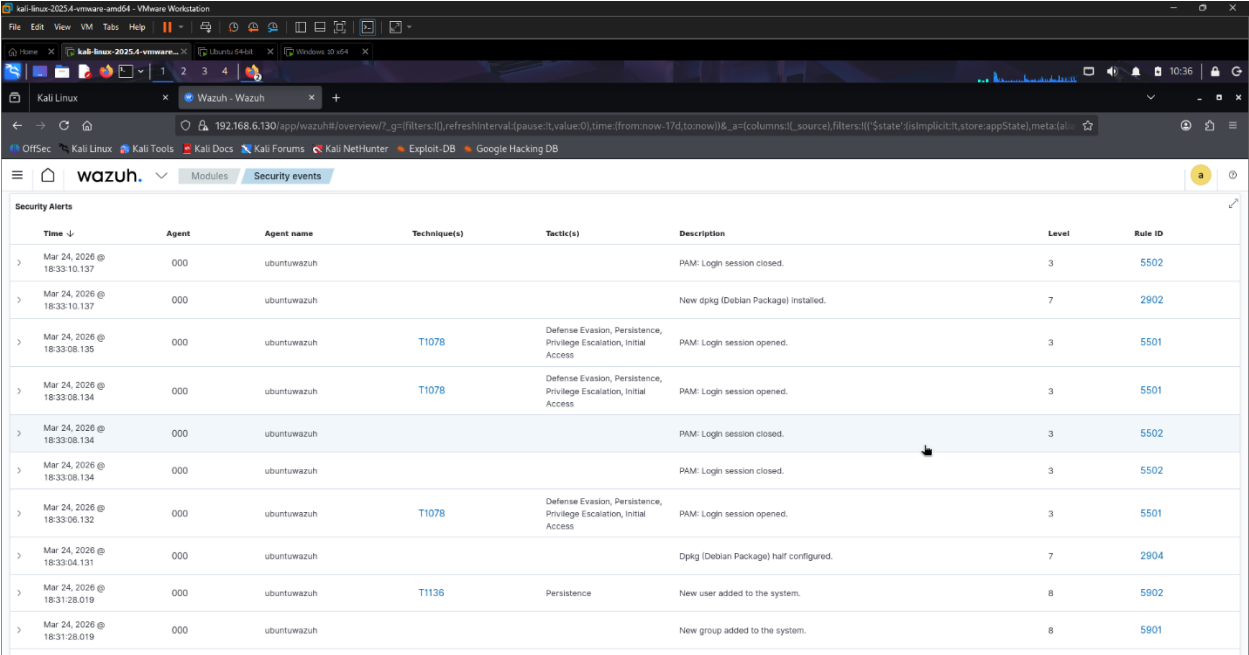
These alerts confirmed successful communication between agents and the Wazuh Manager

Day 7 — Wazuh Dashboard Analysis

The final day focused on analyzing alerts within the Wazuh dashboard.

The Security Alerts section provides detailed information about detected events including:

- Rule ID
- Rule severity level
- Agent source
- Event description



Security Alerts							
Time ↓	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Mar 24, 2026 @ 18:33:10.137	000	ubuntuwazuh			PAM: Login session closed.	3	5502
Mar 24, 2026 @ 18:33:10.137	000	ubuntuwazuh			New dpkg (Debian Package) installed.	7	2902
Mar 24, 2026 @ 18:33:08.135	000	ubuntuwazuh	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Mar 24, 2026 @ 18:33:08.134	000	ubuntuwazuh	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Mar 24, 2026 @ 18:33:08.134	000	ubuntuwazuh			PAM: Login session closed.	3	5502
Mar 24, 2026 @ 18:33:08.134	000	ubuntuwazuh			PAM: Login session closed.	3	5502
Mar 24, 2026 @ 18:33:06.132	000	ubuntuwazuh	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Mar 24, 2026 @ 18:33:04.131	000	ubuntuwazuh			Dpkg (Debian Package) half configured.	7	2904
Mar 24, 2026 @ 18:31:28.019	000	ubuntuwazuh	T1136	Persistence	New user added to the system.	8	5902
Mar 24, 2026 @ 18:31:28.019	000	ubuntuwazuh			New group added to the system.	8	5901

Fig1: Wazuh Security Alerts dashboard displaying detected events, including rule IDs, severity levels, and event descriptions for monitored endpoints.

Wazuh Rule Levels

Alert severity levels range from 0 to 15:

Level Severity

0–3 Informational

4–7 Low

8–11 Medium

12–15 High/Critical

These levels help analysts prioritize alerts.

Alert Triage

Alert triage involves reviewing alerts to determine whether they represent genuine threats or normal activity.

Common alerts observed included:

- Failed SSH login attempts
- Authentication errors
- System log monitoring events

- File integrity checks
- Network connection attempts

The Threat Hunting module was also used to filter alerts by:

- Agent name
- Time range
- Rule level
- Event type

>	Mar 30, 2026 @ 14:10:30.172	ubuntuwazuh	Apparmor DENIED mknod operation.	4	52004
>	Mar 30, 2026 @ 14:10:29.803	ubuntuwazuh	Apparmor DENIED mknod operation.	4	52004
>	Mar 30, 2026 @ 14:10:29.794	ubuntuwazuh	Apparmor DENIED mknod operation.	4	52004
>	Mar 30, 2026 @ 14:10:29.497	Windowwazuh	Windows logon success.	3	60106
>	Mar 30, 2026 @ 14:10:27.738	Windowwazuh	Name resolution for the name wpad timed out	5	61109
>	Mar 30, 2026 @ 14:10:16.885	Kaliwazuh	Listened ports status (netstat) changed (new port opened or c losed).	7	533
>	Mar 30, 2026 @ 13:55:06.528	Windowwazuh	Name resolution for the name wpad timed out	5	61109
>	Mar 30, 2026 @ 13:55:06.528	Windowwazuh	Name resolution for the name wpad timed out	5	61109
>	Mar 30, 2026 @ 13:49:08.456	Windowwazuh	Windows logon success.	3	60106
>	Mar 30, 2026 @ 13:49:08.456	Windowwazuh	Windows logon success.	3	60106
>	Mar 30, 2026 @ 13:44:59.497	Windowwazuh	Name resolution for the name wpad timed out	5	61109
>	Mar 30, 2026 @ 13:44:59.497	Windowwazuh	Name resolution for the name wpad timed out	5	61109
>	Mar 30, 2026 @ 13:33:08.185	Windowwazuh	Windows logon success.	3	60106
>	Mar 30, 2026 @ 13:33:08.185	Windowwazuh	Windows logon success.	3	60106

Fig2 : Threat Hunting interface used to filter and analyze security events based on parameters such as agent, rule level, and time range

The MITRE ATT&CK module was explored to see how alerts correspond to known adversary techniques.

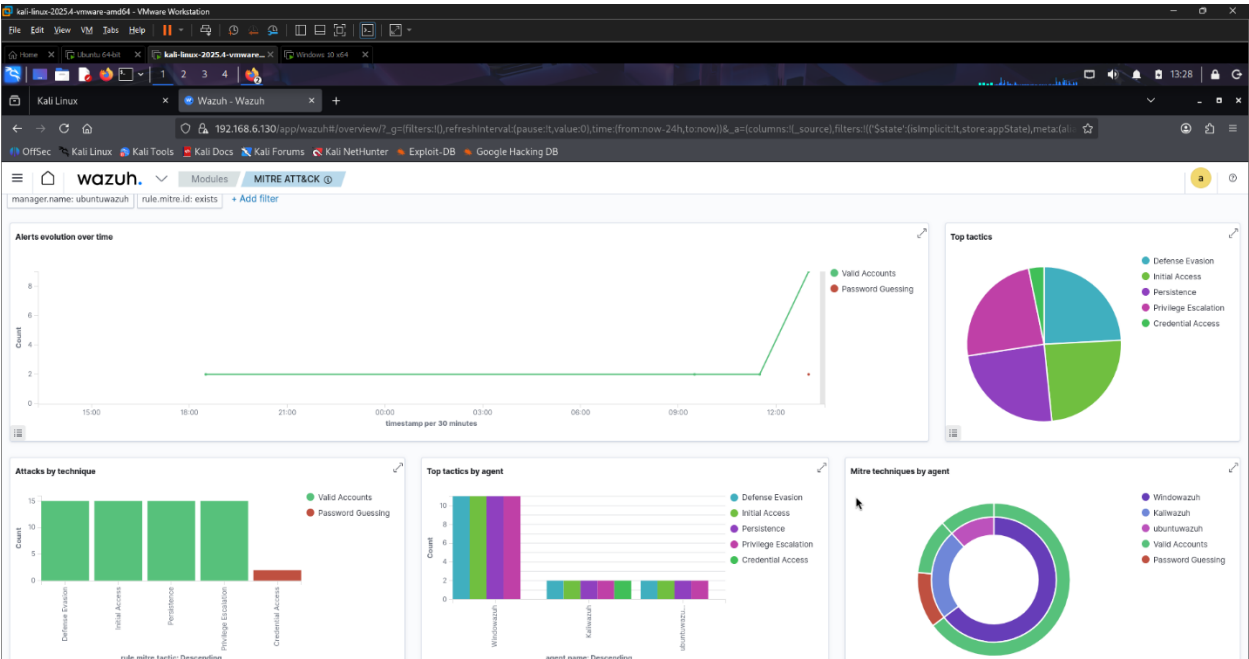


Fig3: MITRE ATT&CK mapping within Wazuh correlating detected alerts to known adversary tactics and techniques

Conclusion

Week 1 successfully established a functional SOC lab environment capable of collecting and analyzing endpoint security events.

The VMware-based environment consisting of Kali Linux, Ubuntu Server, and Windows 10 simulated a small enterprise network. The Wazuh SIEM platform was installed and configured to monitor activity across the environment.

Agents were successfully deployed to monitored endpoints and were able to transmit log data to the Wazuh Manager. Generated test alerts confirmed the monitoring pipeline was working correctly.

Through the Wazuh dashboard, alerts were analyzed and categorized based on severity and context. This process introduced the concept of alert triage, an essential skill for SOC analysts.

This lab environment provides the foundation for the upcoming weeks of the internship, where more advanced security monitoring, detection engineering, and incident response tasks will be performed.